

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1708

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I Lokesh S (192511037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Lokesh S

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. Reverse Engineering a Recommendation System

Introduction

A large-scale e-commerce platform generates accurate product recommendations using continuously updated customer and product data. A likely architecture combines distributed data processing using **PySpark**, RDD-based transformations, a recommendation model, and intelligent agents that monitor user behaviour and update recommendations dynamically.

Reconstructed Architecture

Customers / Website / Mobile App



User Interaction Data

clicks, searches, carts, purchases



Data Collection Layer



Distributed Storage



PySpark / RDD Layer



Data Cleaning & Filtering



Feature Extraction



Recommendation Model



Ranked Products



Recommendation Agent



Website / Mobile Application

Data Collection

The system collects several types of information:

- Product searches
- Page views
- Clicks
- Shopping-cart activity
- Previous purchases
- Product ratings
- Customer preferences
- Time and frequency of interactions

The incoming information can be stored as distributed records so that millions of user interactions can be processed in parallel.

PySpark and RDD Processing

PySpark provides distributed processing over large datasets. An RDD can represent interaction records.

For example:

```
interactions = sc.textFile("user_interactions.csv")
```

A transformation can convert raw data into useful records:

```
records = interactions.map(lambda x: x.split(","))
```

Filtering invalid records:

```
valid = records.filter(lambda x: len(x) >= 4)
```

The system can then create user-product relationships:

```
user_products = valid.map(
```

lambda x: (x[0], x[2])

)

Products frequently interacted with by a user can be grouped and analysed.

Recommendation Generation

The processed data can be used to determine:

- Products frequently purchased together
- Products preferred by similar users
- Products related to a user's browsing history
- Products with high engagement

A recommendation model can calculate a relevance score and rank candidate products.

Intelligent Agent

An intelligent recommendation agent continuously observes customer behaviour.

For example:

User searches for laptop



Agent observes behaviour



Updates user profile



Checks similar products



Generates new recommendations



Displays laptops/accessories

The agent allows the system to react to changing user interests instead of relying only on static recommendations.

Why RDDs Are Useful

RDDs support distributed processing, so large volumes of customer interactions can be divided among multiple machines. They also provide fault tolerance because lost partitions can be recomputed from their lineage.

Conclusion

The reverse-engineered system is therefore likely to use **distributed data collection → PySpark/RDD processing → feature generation → recommendation model → intelligent agent → dynamic recommendation delivery**. This architecture allows the e-commerce platform to process large-scale behavioural data and continuously adapt recommendations.

2. Reverse Engineering a Smart Traffic Management System

The scenario describes a city-wide traffic system that changes traffic signals according to vehicle density and congestion patterns. The system can be reconstructed as a **real-time distributed AI pipeline** with sensors, PySpark processing, and intelligent traffic-control agents.

Reconstructed Architecture

Traffic Cameras / Sensors / GPS



Data Collection



Real-Time Data Stream



PySpark Processing



RDDs



Density + Congestion Features



Decision Model



Traffic Control Agent



Signal Timing Update



Traffic Flow Feedback



Data Collection

The system receives real-time information from:

- Roadside sensors
- CCTV cameras
- Vehicle GPS systems
- Traffic counters
- Junction controllers

The data may contain vehicle counts, average speed, queue length and congestion level.

RDD-Based Processing

Incoming traffic observations can be represented as distributed records.

```
traffic = sc.textFile("traffic_stream")
```

The system can transform raw observations:

```
data = traffic.map(lambda x: x.split(","))
```

Vehicle-density information can be extracted:

```
density = data.map(
```

```
lambda x: (x[0], int(x[2]))
```

)

Values can then be grouped by road or junction.

Decision Making

The system calculates traffic conditions for each junction.

For example:

Vehicle Density = Low

→ Normal signal timing

Vehicle Density = Medium

→ Increase green time slightly

Vehicle Density = High

→ Increase green time for congested direction

Intelligent Agent Interaction

Each traffic junction can be controlled by an intelligent agent.

Junction Agent A

↕

Junction Agent B

↕

Central Traffic Agent

↕

Junction Agent C

Agents exchange information about neighbouring roads. If one road becomes congested, the agent may coordinate with nearby intersections to improve overall traffic flow.

Decisions Under Time Constraints

Traffic control requires low-latency decisions. Therefore, processing must happen quickly:

Sensor Reading



Process



Predict



Decide



Change Signal

The decision cycle may repeat continuously.

Fault Tolerance

RDD lineage allows lost partitions to be recomputed. This is valuable in distributed traffic processing because failure of one processing node should not permanently destroy all collected traffic information.

Conclusion

The smart traffic system likely combines **real-time sensor data, distributed PySpark/RDD processing, congestion analysis, and cooperating traffic-control agents**. The agents use processed information to change signal timing rapidly and optimise city-wide traffic flow.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system that identifies fraudulent transactions within milliseconds requires a highly distributed and low-latency architecture. The likely pipeline uses transaction ingestion, distributed feature extraction, anomaly detection and an intelligent alert agent.

Reconstructed Architecture

Bank Transaction



Transaction Stream



PySpark Processing



RDD Transformation



Feature Extraction



Fraud Detection Model



Fraud Score



Alert Agent



Block / Review / Allow



Customer / Bank System

Input Data

The pipeline may process:

- Transaction amount
- Transaction time
- Customer ID
- Location
- Merchant information
- Device information

- Previous transaction behaviour
- Transaction frequency

RDD Processing

The incoming transaction data can be converted into RDD records.

```
transactions = sc.textFile("transactions.csv")
```

```
data = transactions.map(  
    lambda x: x.split(",")  
)
```

Invalid transactions can be removed:

```
valid = data.filter(  
    lambda x: float(x[2]) > 0  
)
```

Feature Extraction

The system converts raw transaction information into fraud-related features.

Example:

Transaction Amount → unusually high?

Location → unusual location?

Time → unusual transaction time?

Frequency → too many transactions?

Device → unknown device?

These features can be combined into a fraud score.

Anomaly Detection

A detection model identifies transactions that differ significantly from the customer's normal behaviour.

For example:

Normal:

₹500 → ₹1,000 → ₹700

Suspicious:

₹800 → ₹900 → ₹1,50,000

The unusually large transaction could receive a high anomaly score.

Intelligent Fraud Alert Agent

The intelligent agent makes a rapid decision based on the fraud score.

Fraud Score < Threshold

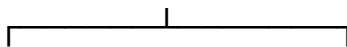


Allow

Fraud Score ≥ Threshold



Alert Agent



Block

Manual Review

The agent can also consider the customer's previous behaviour before generating an alert.

Why Distributed Processing Is Important

Banks may process millions of transactions. A single machine may not be sufficient to handle the workload.

PySpark allows transaction records to be processed in parallel across multiple nodes.

RDDs also provide fault tolerance through lineage, which helps recover lost partitions.

Real-Time Requirement

Because fraudulent transactions must be detected within milliseconds, the architecture should minimise unnecessary processing steps. Feature extraction, model inference and alert generation should be highly optimised.

Conclusion

The likely fraud-detection architecture consists of **stream ingestion** → **RDD-based distributed processing** → **feature extraction** → **anomaly detection** → **intelligent alert agent** → **transaction decision**. Such an architecture provides scalability while supporting rapid fraud identification.

4. Reverse Engineering a Personalized Learning System

The online learning platform adapts content difficulty according to student performance and engagement. The likely design uses learner data, distributed RDD processing, a prediction/decision model and intelligent learning agents.

Reconstructed Architecture

Student



Learning Platform



Clicks / Scores / Time / Attempts



Data Collection



PySpark + RDD Processing



Learner Feature Extraction



Performance Analysis



Learning Agent



Difficulty Selection



Personalized Content



Student Feedback



Learner Data Collection

The system records:

- Quiz scores
- Number of attempts
- Time spent on questions
- Video completion
- Topic progress
- Login frequency
- Incorrect answers
- Engagement patterns

RDD Transformations

The platform can load student activity:

```
logs = sc.textFile("student_logs.csv")
```

Records can be transformed:

```
student_data = logs.map(
```

```
    lambda x: x.split(",")
```

```
)
```

Student-level performance can be grouped:

```
performance = student_data.map(  
    lambda x: (x[0], float(x[3]))  
)
```

The system can then calculate average scores and other learner statistics.

Adaptive Difficulty

Suppose a student consistently obtains high scores:

Score > 80%



Increase difficulty

If performance decreases:

Score < 50%



Provide easier content + additional explanation

This creates personalised learning paths.

Intelligent Learning Agent

The agent continuously observes the learner and decides what content should be provided next.

Student Performance



Learning Agent



Evaluate Skill Level



Choose Next Content



Observe New Result



Update Learner Model

The agent can therefore adapt to each student's changing performance.

Feedback Mechanism

The system can provide feedback such as:

- Additional explanations
- Practice questions
- Revision material
- Easier examples
- Advanced exercises

Scalability

An online platform can have thousands or millions of students. Distributed PySpark processing allows learner records to be processed simultaneously across many computing nodes.

RDDs also provide fault tolerance and allow large datasets to be processed without placing the entire dataset on a single machine.

Conclusion

The reverse-engineered system consists of **student activity collection** → **RDD-based processing** → **learner modelling** → **intelligent learning agent** → **adaptive content selection** → **feedback**. This architecture provides personalised education while remaining scalable for large online platforms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

The cloud platform automatically scales computing resources and detects anomalies in system logs. This system can be reconstructed as a distributed monitoring architecture using **PySpark, RDDs and intelligent agents**.

Reconstructed Architecture

Cloud Servers / Containers



Logs



Log Collection



Distributed Storage



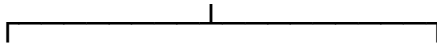
PySpark / RDD



Log Cleaning & Parsing



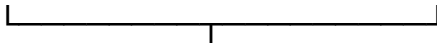
Feature Extraction



Anomaly Detection Resource Analysis



Monitoring Agent Scaling Agent



Cloud Controller



Scale Up / Scale Down



Feedback

Log Processing

Cloud infrastructure produces large amounts of logs, including:

- CPU utilisation
- Memory utilisation
- Network traffic
- Request count
- Response time
- Error rates
- Application logs
- Server events

These logs can be distributed across several machines.

PySpark RDD Processing

The system can load log data:

```
logs = sc.textFile("cloud_logs.txt")
```

It can then filter important records:

```
errors = logs.filter(  
    lambda x: "ERROR" in x  
)
```

Logs can be transformed into structured information:

```
records = logs.map(  
    lambda x: x.split(",")  
)
```

Aggregation can identify resource usage:

```
cpu_usage = records.map(
```

```
lambda x: (x[0], float(x[2]))
```

)

Fault Tolerance

RDDs are fault tolerant because Spark remembers how an RDD was constructed through its lineage. If a partition is lost, Spark can recompute that partition from the original transformations.

This is particularly valuable in cloud environments where failures can occur due to:

- Node failure
- Network failure
- Container failure
- Hardware problems

Anomaly Detection

The platform can identify unusual patterns.

For example:

Normal CPU:

40% → 45% → 50%

Possible anomaly:

40% → 45% → 98%

Similarly, an unexpected increase in network traffic or server errors may indicate an abnormal condition.

Intelligent Monitoring Agent

The monitoring agent observes processed metrics and determines whether intervention is necessary.

Metrics

↓

Monitoring Agent

↓

Normal?

/ \

Yes No

| |

Continue Alert Agent

↓

Investigate

Scaling Agent

The scaling agent manages resources.

Example:

CPU > 80%

↓

Scale Up

↓

Add Resources

CPU < 30%

↓

Scale Down

↓

Remove Unused Resources

This helps maintain application performance while controlling cloud costs.

Agent Coordination

Different agents may have different responsibilities:

Monitoring Agent



Detects anomaly



Alert Agent



Notifies system

Scaling Agent



Checks resource demand



Adds/removes instances

The agents can exchange information so that anomaly detection and resource management occur together.

Conclusion

The cloud monitoring architecture likely uses **large-scale log collection, PySpark/RDD processing, feature extraction, anomaly detection and cooperating intelligent agents**. RDD fault tolerance helps the system recover from distributed failures, while agents enable automatic monitoring, anomaly response and resource scaling.

Overall Conclusion

All five systems demonstrate how distributed AI can combine **PySpark, RDD operations and intelligent-agent architectures** to solve large-scale real-world problems. The architecture generally follows the pattern:

Data Collection



Distributed Storage



PySpark / RDD Processing



Feature Extraction



AI / ML Decision



Intelligent Agent



Real-Time Action



Feedback

PySpark provides distributed computation, while **RDDs** allow large datasets to be transformed and processed across multiple nodes with fault tolerance. **Intelligent agents** add decision-making capabilities by observing system states, selecting actions and adapting to changing conditions.

This combination is useful for recommendation systems, traffic control, fraud detection, personalised education and cloud monitoring because all of these applications require **scalability, continuous data processing, autonomous decisions and distributed execution**. These are the core ideas being assessed under CO5.